Plexus Atlas — CompuCell3D

The second repository, and the first test of whether any of this transfers

1 August 2026

The twin of `atlas_jax_morph/atlas_note.pdf`. That one asked whether Plexus can say what one framework says. This one asks whether the *procedure* is a procedure.

---

1. Why a second repository, and why this one

One repository cannot show saturation. The measurement `plexus2.tex` App. E.1 asks for — *does decomposing a broad collection of frameworks converge on a compact reusable vocabulary?* — is a claim about what happens *across* frameworks. With $n = 1$ the curve is a shape, not an argument.

Target: CompuCell3D (Cellular Potts). Nominated by the jax-morph note before any of this work, on the grounds that it is *the biggest step away from everything we have*: a cell is not a point with per-cell state, it is a **set of lattice sites**. Its position is a derived centroid. Its dynamics are Metropolis flips of site ownership. There is no per-cell equation to subtract.

That is the point. Every measurement in the jax-morph atlas rested on cells being points and a differential test being a subtraction. **If the procedure is general it survives this; if it is secretly a particle-framework procedure, this is where it breaks.**

The prediction — made before running anything, and the reason this note exists — is in `TRANSFER.md`.

2. What transferred, measured rather than asserted

`STATUS.md` in the jax-morph atlas claimed ∼60% of the Python was instrument and would copy as-is. Forking it is the test of that claim, and it came out slightly better than claimed.

file	edits	why
`record.py`	none	schema, ladder and the twelve rules are about the record, not the target. Self-test passes unchanged.
`saturation.py`	none	to *transfer*: the ledger scores against the frozen baseline and nothing in it knows what a cell is. It was later *extended* — see §7 — but that is a defect the second repository exposed, not a porting cost.
`atlas.py`	none	blast radius, rung, commit-or-revert — all `HERE`-relative.
`registry_view.py`	none	reads `plexus.operators`. Produced the same 52-contract baseline from the new folder.
`agents/`	none	the six roles describe a job, not a repository.
`run_spec.py`	5 lines	output paths only (`log/atlas_cc3d/`, `graphs_data/atlas_cc3d/`).
`oracle.py`, `inventory.py`, `paper.py`	rewrite	target-specific by construction: the reference, its mechanism scan, its PDF.

Five of the seven instruments needed zero edits. That is the strongest evidence so far that the apparatus is a method rather than a one-off, and it cost about ten minutes to establish.

3. The oracle — where the risk actually is

`TRANSFER.md` predicted the oracle would be the largest risk, and a *target* risk rather than a *procedure* risk. It was both, and **it now passes**.

gate	state	
installs	**done**	CompuCell3D 4.10.0 from its own conda channel into an isolated environment (`/workspace/.conda_envs/cc3d-oracle`), py312.
imports	**done**	after supplying `libGL` — the container is headless and CompuCell3D links its graphics stack unconditionally.
model definable in Python	**done**	`cc3d.core.PyCoreSpecs`, so the model is written in Python and never hand-typed as XML.
isolation	**done**	the Plexus interpreter *cannot* `import cc3d`. Checked, not assumed: a process that can reach the reference can borrow its answer.
runs headless	**done**	9 cells at 30^2, 45 at 60^2; artefacts with provenance in `_oracle/runs/smoke/`.
deterministic at a fixed seed	**done**	two runs at seed 42 are *bit-identical* across id, type, volume, surface and centre of mass; seed 7 differs.

Getting there meant routing around three defects in CompuCell3D 4.10.0, none of them configuration:

1. The advertised pure-Python route (`PyCoreSpecs` → `service_cc3d`) passes the *list of specs* where a simulation file is expected: `get_custom_settings_path()` does `Path(<list>)`. Shimming that only exposes the next one — `CC3DSimService._run` asserts `os.path.isfile(<list>)`. **That route simply does not work in this release.**

2. CC3D `execs` the project's Python script with its own globals, so a steppable class defined in the same file raises `NameError: SteppableBasePy is not defined` at registration. The class must live in a separate importable module.
3. The output directory may not sit inside the project folder.

The way through is to use `PyCoreSpecs` for what it is genuinely good at — generating correct CC3DML, `<RandomSeed>` included — write a real `.cc3d` project, and run it through `cc3d/run_script.py`, the officially supported headless entry point, which never touches the broken path. The model stays defined in Python; only the transport is a file.

Why this mattered enough to spend the time on. jax-morph was chosen *because* it ran, and the first note said so in its own Phase 0. A target that cannot be run reduces the atlas to the Okuda situation — wrong operator, wrong parameter or wrong reading of a figure, with no way to tell which. **Determinism is the gate behind the gate**: without it, every later disagreement is unattributable between their noise and our error. Both now hold, so Phase 1 is unblocked.

4. The phases

Seven stages, the same ladder as the first atlas. **I stop at the end of each one and wait for you.** What differs is not the procedure but what each stage has to mean when a cell is a region of a lattice rather than a point.

Phase 0 — Instruments and the oracle DONE

Goal. Make every later measurement trustworthy before taking one.
The instruments came across for free (§2). **The oracle was the phase**, and it is where the risk always was: install, import, isolate, run headless, and be deterministic at a fixed seed.
All six gates now pass (§3), including the stop condition — the authors' own cell-sorting model runs twice from one seed and is bit-identical. Until that held, a later disagreement would have been unattributable between their noise and our error. It holds.

Phase 0 — what actually happened DONE

	state	the issue found, and what was interesting
instruments	done	Five of seven needed **zero edits** — `record.py`, `saturation.py`, `atlas.py`, `registry_view.py`, `agents/`. Both self-tests pass from the new folder and the same 52-contract baseline is reproduced, so the two campaigns are scored against one ruler.
oracle installs	done	CompuCell3D 4.10.0 from its own conda channel, isolated env, py312.
oracle imports	done	needed `libGL`: the container is headless and CompuCell3D links its graphics stack unconditionally.
isolation	done	the Plexus interpreter cannot `import cc3d`. Checked, not assumed.
headless run	done	**Three upstream defects had to be routed around.** The advertised pure-Python route (`PyCoreSpecs` → `service_cc3d`) passes the specs *list* where a file is expected — `Path(<list>)`, then `os.path.isfile(<list>)`. CC3D `execs` the project script with its own globals, so a steppable class in that file raises `NameError` at registration. And the output directory may not sit inside the project. The way through: generate CC3DML *from* `PyCoreSpecs`, write a real `.cc3d` project, run `cc3d/run_script.py`.
determinism	done	two runs at seed 42 bit-identical across id, type, volume, surface and centre of mass; seed 7 differs. **The first attempt compared a dump whose `xCOM/yCOM` were all zero** because `CenterOfMassPlugin` was not enabled — a weaker test than it looked, so the plugin was added and the check re-run.

And a diagnostic mistake worth recording. Two runs appeared to produce *no output* and were reported as hangs at import. They were not: piping an unbuffered process through `grep` buffers its output, which is lost when the process is killed on timeout. The marks existed; the pipe ate them.

Phase 1 — Read every mechanism DONE

The excavator on each mechanism: what it does to the state, its equations, the role of every knob, and the `surprises` a reimplementer gets wrong.

What is different here. jax-morph's mechanisms were 20 classes plus 4 architectural contracts a scan could not see. CompuCell3D's are *plugins* and *steppables* declared through XML or `PyCoreSpecs` — a different extraction problem, and one where the scan-versus-hand split will fall in a different place. **The 4 hand-added entries were among the most interesting things the first atlas found**; expecting an automatic scan to be sufficient here would be repeating a mistake we already know about.

Phase 1 — what actually happened DONE (35 of 35)

The inventory is complete and it is bigger than the first atlas's: 35 mechanisms, against jax-morph's 24. Thirty come from `cc3d.core.PyCoreSpecs` — the framework's *own* registry of what a model can switch on, which is a better source than a syntax-tree scan because it is the project's index rather than our reading of it.

Five were added by hand, and they are the interesting ones. The first atlas found that 4 of its 24 were not classes; here the same is true and more so, because a Cellular Potts model's defining commitments are not plugins:

`cell_as_lattice_domain`	a cell is a set of lattice sites sharing an id; volume is a count, position a derived centroid
`metropolis_acceptance`	$P = 1$ if $\Delta E \leq 0$ else $e^{-\Delta E/T}$ — discrete, stochastic, and with no pathwise derivative
`energy_sum_composition`	mechanisms compose by *adding energy terms*, not by operator splitting. The direct counterpart of jax-morph's Lie–Trotter split, and the contrast is itself a finding
`mcs_time_unit`	one MCS is one attempted copy per site. **There is no `dt`**
`pixel_neighbourhood`	the model's relation is a lattice adjacency, not an edge set built from positions

All 35 are now at *inspected*: 0 candidates, 0 validator violations, 0 blocked. The excavator read each mechanism at source in the installed package — 25 agent calls at roughly 4 minutes each, none reverted for a bad verdict.

The prompts had to be retargeted first, and that is the part worth recording. The forked `agents/` still described jax-morph: its paper, its extracted text, its guides directory. Running it unchanged would have sent 24 agents looking for `papers/jax-morph`. The preamble now describes a Cellular Potts framework, says where the source actually is (an absolute `file:Lnn` into the installed package), states plainly that **we have no extracted paper text** — "do not cite page numbers you have not read" — and adds a rule the first campaign never needed: *most CC3D mechanisms are energy terms, not updates; say so in `state_io` rather than forcing them into read/write language that does not fit.*

The record is checkable. `R3_code_path` resolves for every entry because each carries a real `file:line`. The first seeding used dotted module names and R3 rejected all 11 — correctly, and that is the rule doing its job rather than a nuisance.

Eight mechanisms were BLOCKED and then unblocked, with the reason logged. Three agents changed nothing (a call that produces no record change produced nothing), one wrote YAML that stopped parsing, one timed out, and two touched `atlas_record.yaml` outside the driver and were restored. Every one was completed on a later call. A stale block is its own kind of lie — `status` keeps reporting work outstanding that is done — so each clearance is written to `_state/unblocked.jsonl` with the evidence that justified it (equations, `state_io`, and a note file all present), rather than quietly deleted.

The excavations are real work, not summaries. `diffusionsolverfe`'s update is transcribed from the OpenCL kernel at L954–L1054; `curvature` is identified as the Menger curvature $\kappa = 2\sin\theta/|R - L|$ of three junction-linked centres of mass; `contactlocalflex` is traced through the compiled core to a per-cell `ContactLocalFlexDataContainer` that the identical Python constructor does not reveal.

What the six runs already establish, each with its control (see §5): every CPM mechanism here is an *energy term*, not an update. None of them writes state; they change only the probability that a proposed pixel copy is accepted. That is a structural fact about the whole framework, and it is what Phase 2 will have to normalize against a language whose operators return deltas.

Phase 2 — Normalize, and let the skeptic attack DONE

Each mechanism becomes a typed Plexus contract with a verdict, then the skeptic tries to break it.

The prediction is already on the record (`TRANSFER.md`): the *biological* mechanisms — growth, division, death, chemotaxis, secretion, diffusion — should come back largely `alias` or `implementation` of contracts the first atlas already registered, while the *representational* ones — cell-as-domain, the Metropolis acceptance rule, contact energy over shared boundary, volume and surface constraints — should be genuinely new. If that splits as predicted, the conclusion is that *the biological vocabulary is converging while the representational vocabulary is not.*

Phase 2 — what actually happened DONE (35 normalized, 14 challenged)

The measurement, against the same frozen 52-contract baseline the first atlas used:

	jax-morph	CompuCell3D
mechanisms scored	18	24
alias	2	4
refinement	2	6
implementation	6	6
out of scope	6	11
genuinely new	**8**	**8**
yield (new per scored mechanism)	0.44	**0.33**

The second repository yields FEWER new contracts per mechanism than the first — 0.33 against 0.44 — which is the direction the saturation hypothesis predicts and was not guaranteed. It is one data point, not a curve.

The cross-repository fix earned itself immediately. Without `--prior`, CompuCell3D scores **9** new contracts and re-counts `adhere` as a first sighting. With it, the four contact plugins (`Contact`, `ContactLocalFlex`, `ContactInternal`, `AdhesionFlex`) are correctly classified as *implementations* of the `adhere` contract jax-morph already proposed. **One contract, double-counted, is a 12% error on the headline** — and it would have grown with every further repository.

The normalizer found the strongest case itself. On the last mechanism it reported that CompuCell3D's `SteadyStateDiffusionSolver` solves *the identical equation* as jax-morph's `morphogen` — $D\nabla^2 c - kc + S = 0$ — and classified it as a second sighting rather than new vocabulary. That is the convergence claim showing up as a concrete match between two unrelated codebases, not as an assertion.

What is genuinely new here is representational, as predicted. `occupy` — a cell as a set of lattice sites — is new, and so are `volume_elasticity`, `membrane_tension`, `elongate`, `stiffen`, `bond`, `stay_connected` and `react`. **The prediction in `TRANSFER.md` was half right**: the biological mechanisms did collapse into contracts the first atlas already had, and the representation did yield something new. But three of the five architectural entries — the Metropolis rule, the energy-sum composition and the MCS time unit — came back *out of scope* rather than new: the normalizer judged them framework mechanics with no biology, exactly as jax-morph's four architectural entries were judged. **Two frameworks, two very different architectures, and both times the architecture scored zero new biological vocabulary.**

The skeptic earned its place again. Every `new` claim was challenged — 14 mechanisms, since the record held more `new` verdicts than the ledger counts distinct contracts. **Five were refuted and sent back to the normalizer**: `adhesionflex`, `connectivity`, `lengthconstraint`, `surface` and `volume`. `connectivity` was demoted outright from `new` to `alias`. Nine survived, at confidences of 0.65–0.85.

And a driver lesson worth recording. The first attempt ran `step --role normalizer --skeptic` over the already-normalized entries, which re-runs the NORMALIZER first — and a call that changes nothing counts as a failure. Nothing was corrupted (the guard rejected the no-op edits and the measurement was unchanged), but only one challenge actually landed. `--skeptic` belongs *during* the normalizing pass; afterwards, the skeptic must be invoked as its own role.

Eleven out-of-scope is nearly half the framework, against six of twenty-four for jax-morph. A Potts framework carries far more machinery that models no biology — trackers, initialisers, dumpers, the acceptance rule itself.

Phase 3–7 — DEFERRED, deliberately NOT STARTED

The order changed, and this is the reason. The original plan ran all seven phases per repository: read, normalize, implement, validate, figure, differentiate, promote. That is the right order for *one* target and the wrong order for the programme.

Extraction is cheap and reversible; committing the language is not. Phases 1–2 only read and classify — they write a record and nothing else. Phase 3 writes operator modules into `src/plexus/operators/candidates/`, and Phase 7 promotes them into the language proper. Implementing CompuCell3D's sixteen candidates now would shape the anti-chamber around *one* framework before we know what the next three need — and the catalog is precisely the instrument that tells us that.

So the revised programme is:

1. **Extract** (Phases 0–2) across many repositories. Each yields a record and a contribution to the ledger. Cheap, parallel, and nothing is committed.
2. **Catalog** (`catalog.py`) — merge every record, count each contract ONCE across repositories, and read the cumulative-new curve. This is the only place the App. E.1 claim can actually be tested.
3. **Then** implement, validate and promote — on the contracts the catalog says survive contact with more than one framework.

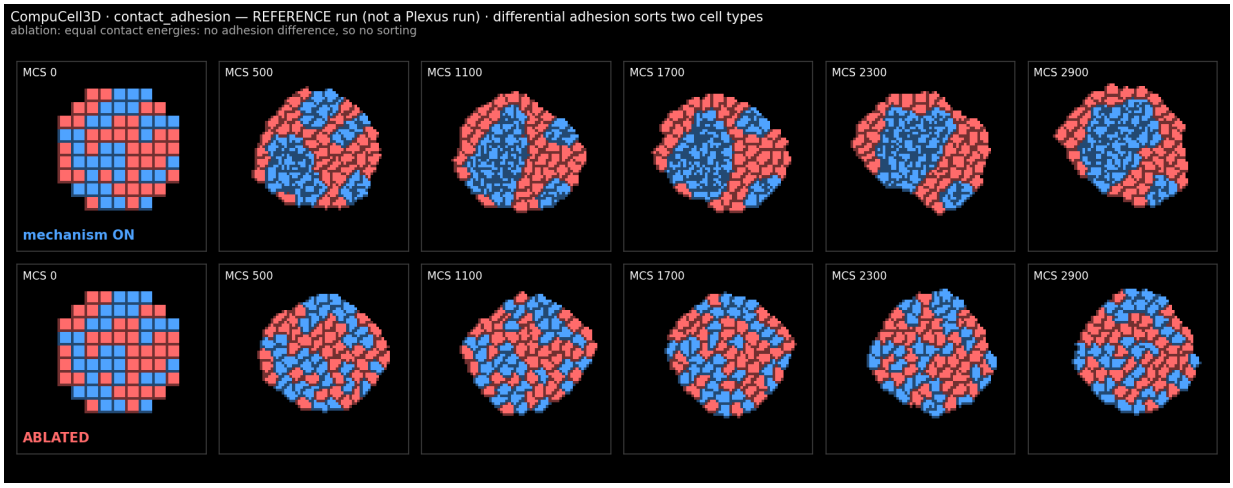
What defers with them. Phase 4's differential tests need implementations, so they wait too. Phase 6 (differentiability) was in any case likely inapplicable here — a Metropolis accept/reject has no pathwise derivative — and saying so is a finding rather than a gap.

What this costs. The record's `status` ladder will sit at *normalized* for every repository, and the ledger's verdicts stay unvalidated against the reference. That is a real limitation and it is the price of not guessing: **a verdict of 'new' that no differential test has checked is a claim about our reading of the source, not about the source.**

5. Six mechanisms, each beside its ablation

Reference runs, not Plexus runs — Phases 2–4 have not started, and every artefact says so on its face. They exist to prove the oracle is real and readable, and to put each mechanism's observable in the currency Phase 4 will need.

mechanism	observable	mechanism ON	ablated
`contact_adhesion`	heterotypic boundary	290 → 167 (-42%)	290 → 388 (+34%)
`volume_constraint`	mean volume	25 → 59.4 (+138%)	25 → 0 (-100%)
`surface_constraint`	mean perimeter	20 → 19.3 (-3%)	20 → 22.8 (+14%)
`chemotaxis`	mean x (source at wall)	15 → 18 (+20%)	15 → 14.8 (-1%)
`growth_mitosis`	live cells	37 → 296 (+700%)	37 → 37 (0%)
`external_potential`	mean x	32 → 57.6 (+80%)	32 → 32.2 (+0.5%)



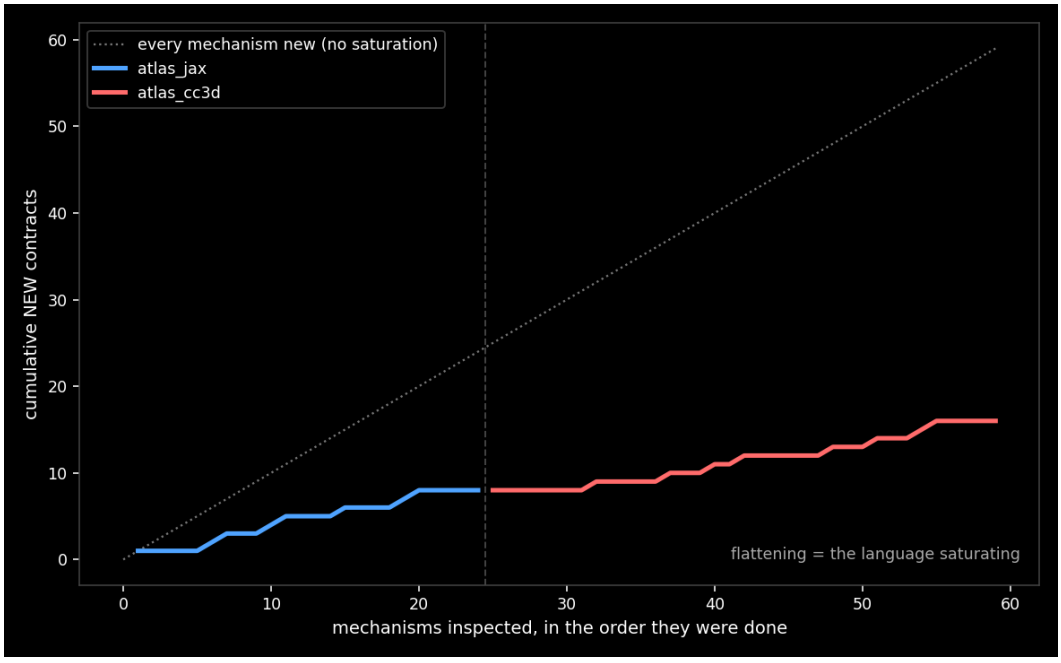
Cell sorting, mechanism above its ablation on matched frames. Colour is cell type; the seams are cell borders drawn from the id lattice, because colouring by type alone renders a one-type tissue as a solid block and makes the cells — the entire subject of a Potts model — invisible.

Every ablation separates, and several reverse rather than merely weaken: sorting becomes mixing, the perimeter grows instead of shrinking, and without a volume constraint the cells do not just relax — they dissolve.

Every observable is a population or topology statistic, never a trajectory. That is forced rather than stylistic. A Potts model is a Metropolis chain, so a matched trajectory is not a meaningful object and Phase 4 will have to compare distributions — the same argument that made the first atlas's division differ compare a pooled *hazard* instead of cell counts, now applying to an entire framework.

Four defects were fixed rather than shipped, each found by looking at the output: type-level volume parameters silently disabling per-cell growth; type-colouring hiding every cell; a chemotaxis strength that transported cells into the wall and destroyed them; and the `PyCoreSpecs` API not being spelled like CC3DML.

6. The catalog — where the claim is actually tested



repository	mech	scored	new	impl	alias	refine	yield
`atlas_jax`	24	18	8	6	2	2	0.44
`atlas_cc3d`	35	24	8	6	4	6	**0.33**
total	59	42	**16**				

Two repositories, 59 mechanisms, **16 distinct new contracts**. The curve sits far below the diagonal on which every mechanism is new vocabulary, and it flattens *within* each repository — but two points do not make a trend, and the honest reading is that the instrument now exists and the measurement has begun.

The number that matters is not 16, it is 5. Five contracts have been sighted in a repository other than the one that introduced them:

CompuCell3D mechanism	contract	first seen in
`Contact`, `ContactLocalFlex`, `ContactInternal`, `AdhesionFlex`	`adhere`	jax-morph
`SteadyStateDiffusionSolver`	`morphogen`	jax-morph

A contract that only ever appears in the repository that introduced it is a description of that repository. One that reappears independently is evidence the vocabulary is real. The `morphogen` case is the sharpest: two unrelated codebases, one written in JAX around point particles and one a C++ lattice framework, solving the same equation $D\nabla^2 c - kc + S = 0$ — and the normalizer identified the match itself.

What would refute the programme is the curve failing to flatten as repositories are added: if each new framework keeps contributing 8 contracts nobody else needs, the algebra is cataloguing our naming habits rather than converging on biology. That is measurable, and it is what the next few extractions are for.

7. An instrument defect only the second repository could reveal — fixed

`saturation.py` classifies a mechanism `implementation` when it reaches a contract *this run* has already counted as new. At $n = 1$ that is complete. At $n = 2$ it is wrong: every contract the first atlas introduced would be counted as new *again* by the second, inflating precisely the number the instrument exists to protect.

The fix is a `--prior` flag: an earlier `atlas_record.yaml` pre-seeds the "already spoken for" set, so a second sighting reads as an implementation *of that repository* rather than as new vocabulary. Checked by feeding jax-morph's record back to itself — its 8 new contracts all demote and it scores **NEW 0, implementation 14**, which is the correct answer for a repository measured against itself.

Applied to *both* campaigns' copies, so the instrument stays one instrument. That is the whole value of a second target arriving before the first is promoted: **the defect was invisible at $n = 1$ and structural at $n = 2$**, and it would have silently doubled the headline number.

Where things are. `TRANSFER.md` — the prediction, written first. `../atlas_jax_morph/STATUS.md` — what the first atlas produced and what was reusable. `_state/registry_baseline.json` — the same frozen 52 contracts, so the two measurements are comparable.